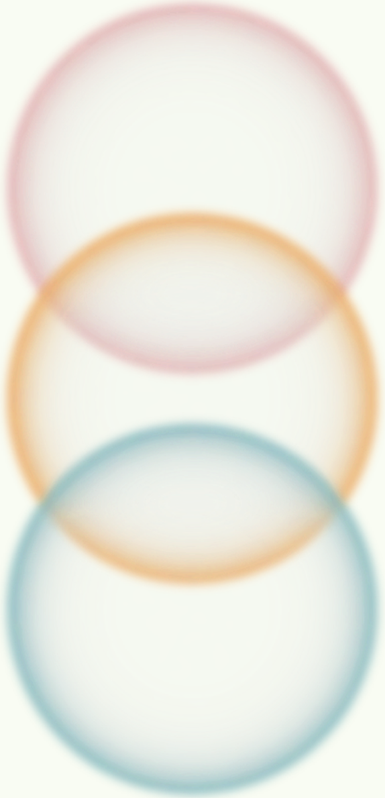
# Diseño

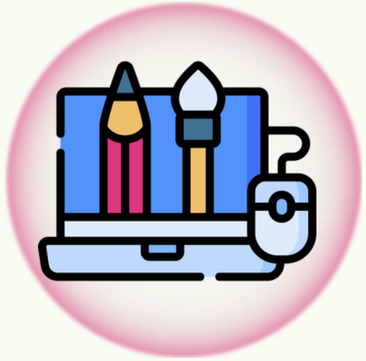
**Instructor Manuel Porras**
mf.porras@uniandes.edu.co

¿Qué tienen que
hacer esos roles?



Esos roles tienen unas **responsabilidades**. ¿Recuerdan?

# Responsabilidades: ¿Qué hacen los participantes?



**Conocimiento:** Un elemento debe **saber o contener** algo.

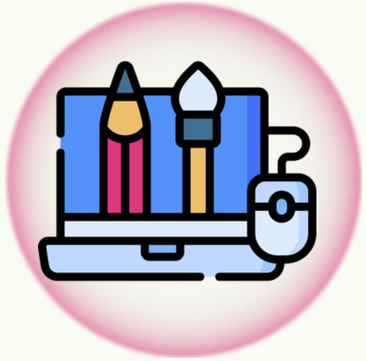


**Acciones:** Indican que un elemento debe ser capaz de **hacer** algo.



**Decisiones:** Indican que un elemento debe ser capaz de tomar una **decisión**.

# Responsabilidades: ¿Qué hacen los participantes?



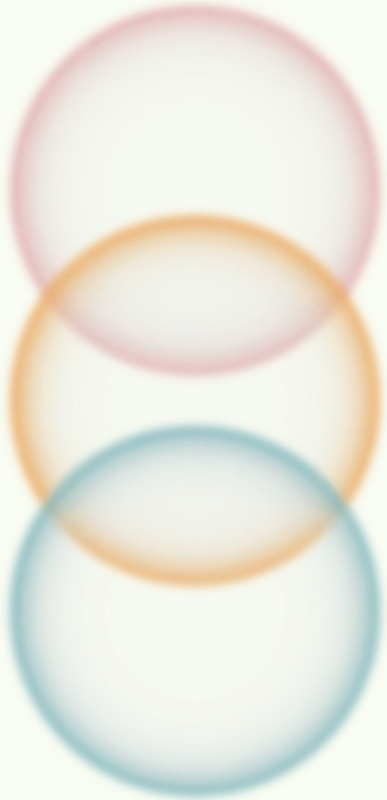
**Conocimiento:**
TransferenciaBancaria probablemente tendrá que saber entre qué cuentas se hizo la transferencia, cuándo se hizo y cuál fue el monto.



**Acciones:**
ValidadorSaldo posiblemente tendrá entre sus responsabilidades validar el saldo de una cuenta.

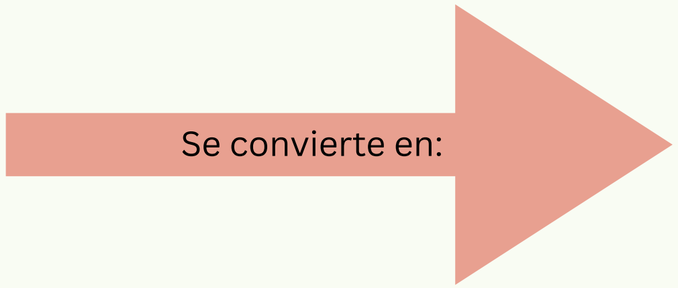


**Decisiones:**
SelectorFormato parece que tiene entre sus tareas tomar decisiones con respecto al formato de algo.



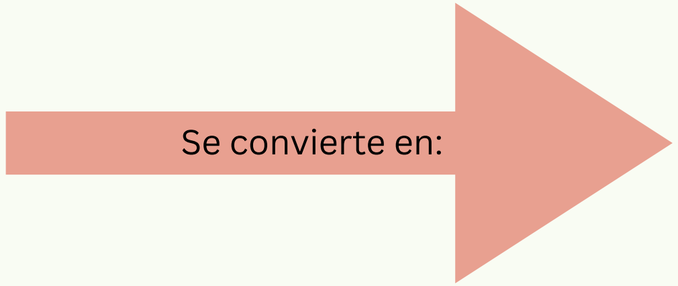
Las responsabilidades a las que se comprometa un elemento tendrán que verse reflejadas en el **contrato** de ese elemento.

Cuando:

**Elemento** 

**Objeto**

Luego:

**Contratos** 

**Métodos**

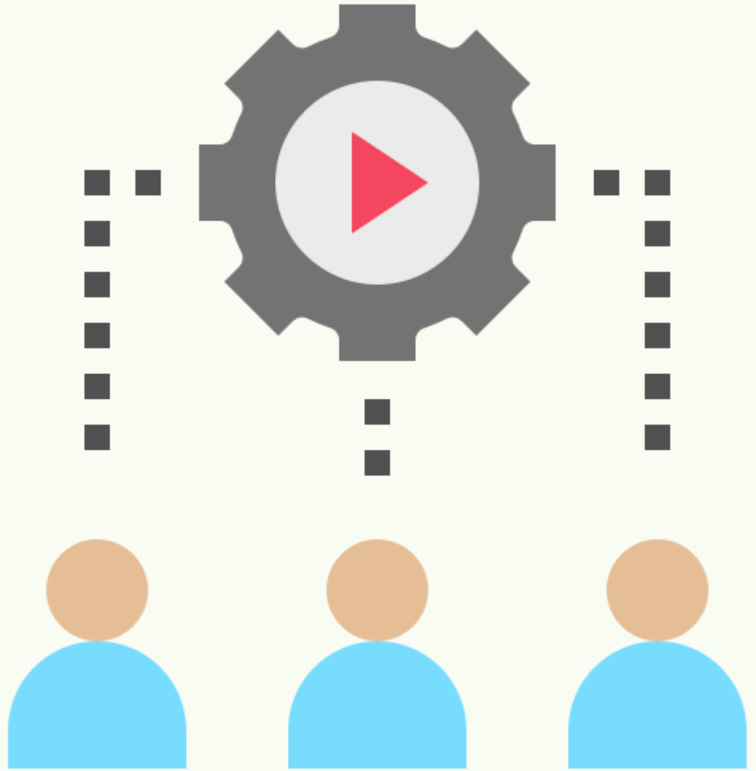
# Estrategias para descubrir responsabilidades

1. Identificar responsabilidades del sistema mencionadas en los requerimientos.
   - Verificar que el saldo de una cuenta es suficiente para hacer una transferencia
2. Encontrar *huecos* en las descripciones (cosas implícitas) o responsabilidades técnicas necesarias para poder cumplir con los requerimientos.
   - Descontar el 4x1000 al hacer la transferencia
3. Extraer comportamiento relacionado que sea relevante dentro del dominio.
   - Registrar la transferencia en el historial de la cuenta origen y la cuenta destino
4. Seguir cadenas de razonamiento del tipo: "Si pasa … entonces …"
   - Si la cuenta destino está bloqueada entonces no se debe poder hacer la transferencia.
   - Alguien debe verificar si una cuenta está bloqueada antes de intentar la transferencia
5. Identificar responsabilidades basadas en los estereotipos de los elementos
   - Almacenar la información sobre una transferencia (relacionado con el estereotipo InformationHolder)
6. Buscar responsabilidades relacionadas con mantener las estructuras y relaciones entre elementos.

7. Identificar responsabilidades relacionadas con el ciclo de vida de los elementos.
   - Alguien debe ser capaz de conseguir la información para poder construir objetos de tipo Cuenta que se puedan manipular durante las validaciones y las transferencias.
8. Buscar otras responsabilidades técnicas necesarias para que el sistema pueda funcionar dentro de su contexto.
   - Enviar notificaciones por SMS a los dueños de las cuentas cuando se realice la transferencia.

Listo, tenemos las responsabilidades, ahora ¿Quíen las hace?

# Toca repartirlas entre los roles



# Volvamos a nuestro caso de estudio y apliquemoslo

# Caso de estudio: actividades

1. Identificar responsabilidades candidatas
2. Asignar responsabilidades
3. Escribir descripciones de cada responsabilidad (no se muestra)

# Responsabilidades candidatas

Decidir qué página ejecutar según la ruta
Decidir qué archivo entregar según la ruta
Decidir si es un archivo o una página
Almacenar y consultar información en una sesión
Crear una sesión
Hacer un query a la BD
Abrir una conexión a la BD
Reconectar con la BD
Ensamblar el resultado de una página
Ejecutar un query sobre la BD
Solicitar que se llene una plantilla
Conseguir la información para llenar una plantilla
Buscar y devolver un archivo local

1. ¿Qué cosas tiene que hacer la aplicación?

Crear cookies
Crear petición
Crear respuesta
Crear una plantilla llena

2. Ciclo de vida de los elementos

Crear el Enrutador
Crear el Manejador de Sesiones
Crear el ConectorDB
Crear el Buscador Archivos
Crear Aplicación

3. Mantener la estructura de los elementos que no se crean por cada petición

Mantener información

4. Mantener y organizar la información sobre el estado del sistema

- Estos son los primeros grupos de responsabilidades candidatas que se identificaron para el sistema.
- Los grupos corresponden a la perspectiva o estrategia utilizada para identificarlos (¿Qué estábamos pensando cuando se nos ocurrió que alguien debería tener esta responsabilidad?).
- La perspectiva 4 es la que tiene ver con las responsabilidades de cada componente para mantener su propia información.
- Estas agrupaciones no tienen ninguna trascendencia de acá en adelante – sirven sólo para mostrar que hay que mirar el problema desde varias perspectivas para encontrar más responsabilidades

# Asignar responsabilidades

| No | Responsabilidad | Componente |
|---|---|---|
| 1 | Decidir qué página ejecutar según la ruta | Enrutador |
| 2 | Decidir qué archivo entregar según la ruta | Enrutador |
| 3 | Decidir si es un archivo o una página | Enrutador |
| 4 | Almacenar y consultar información en una sesión | Página |
| 5 | Crear una sesión | Manejador de Sesiones |
| 6 | Hacer un query a la BD | Página |
| 7 | Abrir una conexión a la BD | ConectorDB |
| 8 | Reconectar con la BD | ConectorDB |
| 9 | Ensamblar el resultado de una página | Página |
| 10 | Ejecutar un query sobre la BD | ConexiónDB |
| 11 | Solicitar que se llene una plantilla | Página |
| 12 | Conseguir la información para llenar una plantilla | Página |
| 13 | Buscar y devolver un archivo local | BuscadorArchivos |
| 14 | Crear cookies | Página |
| 15 | Crear petición | Interfaz HTTP |
| 16 | Crear respuesta | Página |
| 17 | Crear una plantilla llena | Máquina de Templates |
| 18 | Crear el enrutador | |
| 19 | Crear el Manejador de Sesiones | |
| 20 | Crear el ConectorDB | |
| 21 | Crear el Buscador Archivos | |
| 22 | Crear Aplicación | |

- Ahora le asignamos las responsabilidades candidatas a los diferentes componentes candidatos que tenemos con base en su nombre y en el estereotipo de cada uno (*controlador, service provider*, etc.) .
- Dentro de estas responsabilidades, descubrimos que algunas están asignadas a elementos que están por fuera del componente que estamos diseñando (Interfaz HTTP y Máquina de Templates). Esas responsabilidades las dejaremos de lado y las tendremos en cuenta cuando diseñemos esos otros componentes.
- También tenemos un grupo de responsabilidades que no tienen un componente asignado. Eso nos lleva a la necesidad de agregar un rol adicional a nuestro conjunto de candidatos: **CoreContenedor**.
- Con respecto a las responsabilidades de manejo de información, la asignación es sencilla: cada elemento con el estereotipo Information Holder es responsable de manejar su propia información.
- Eso nos deja, sin embargo, con la pregunta: ¿Quién es responsable de saber a qué página corresponde cada ruta? La asignación que realizamos es la siguiente:

  El enrutador es responsable de saber a qué aplicación corresponde cada ruta.

  Las aplicaciones son responsables de saber a qué página corresponde cada ruta.

Ahora, miremos como colaboran los roles entre si

# Colaboraciones

Las responsabilidades de una aplicación entera son asumidas por múltiples elementos **trabajando en conjunto**: cada uno tiene sus propias responsabilidades, pero rara vez se lograrán los objetivos superiores sin que haya interacción entre ellos.



- El diseño de las colaboraciones (y la asignación de responsabilidades) define el estilo de control de una aplicación (o de una parte de una aplicación):
- Algunos objetos colaboran mucho; otros colaboran muy poco.

Hagan de cuenta que funciona como en un vecindario-barrio-conjunto

# Estrategias para diseñar colaboraciones

1. Revisar los estereotipos

   - Information holders: ¿De dónde sacan la información? ¿Persiste la información? ¿Quién maneja la persistencia? ¿Hay algún tipo de caché? ¿Hay que convertir entre formatos?
   - Structurers: ¿De dónde salen los elementos que hay que estructurar? ¿Hay que crearlos, solicitarlos o recibirlos? ¿Cómo se accede a los objetos de la estructura? ¿Qué relación hay entre los objetos y el structurer?
   - Service-providers: ¿Quién tiene la información que necesita el service-provider? ¿Es configurable? ¿Hay varias versiones del mismo servicio dentro de la aplicación?
   - Controladores: ¿Quién tiene la información que el controlador necesita para tomar decisiones? ¿Hasta dónde llegan las responsabilidades del controlador? ¿Las decisiones son suficientemente complejas como para repartirlas?
   - Coordinadores: ¿Cómo delega trabajo un coordinador? ¿Cómo conoce a los elementos a los que tiene que delegarles el trabajo?
   - Interfacers: ¿Qué otros elementos necesita conocer el interfacers? ¿Cómo interactúan con el interfacer otros elementos de la aplicación? ¿Cómo el interfacer conoce y se conecta con los elementos externos con los que tiene que interactuar?

# Estrategias para diseñar colaboraciones

1. Revisar los estereotipos
2. Revisar las responsabilidades (saber, hacer, decidir)
3. Detallar responsabilidades complicadas para encontrar elementos más pequeños que sean más sencillos de distribuir y coordinar
4. Diseñe las colaboraciones para una funcionalidad muy importante y use eso como base para las siguientes.
5. Identifique patrones y utilícelos
6. Prefiera pocas interacciones gruesas (significativas, mensajes con información complejas) que muchas interacciones sencillas (intercambio de información de bajo nivel, solicitudes pequeñas y puntuales)
7. En lo posible, mantenga las cosas dentro del vecindario
8. Intente simplificar

Ahora, volvamos al caso de estudio y apliquemos esto

# Caso de estudio: actividades

1. Identificar colaboraciones y distinguir entre las críticas y las no tan críticas
2. Diseñar las colaboraciones y analizar el estilo de control
3. Ajustar las responsabilidades para que coincidan con las colaboraciones

# Colaboraciones principales

Hay 4 escenarios principales en los cuales se ve la colaboración entre los componentes de la solución:

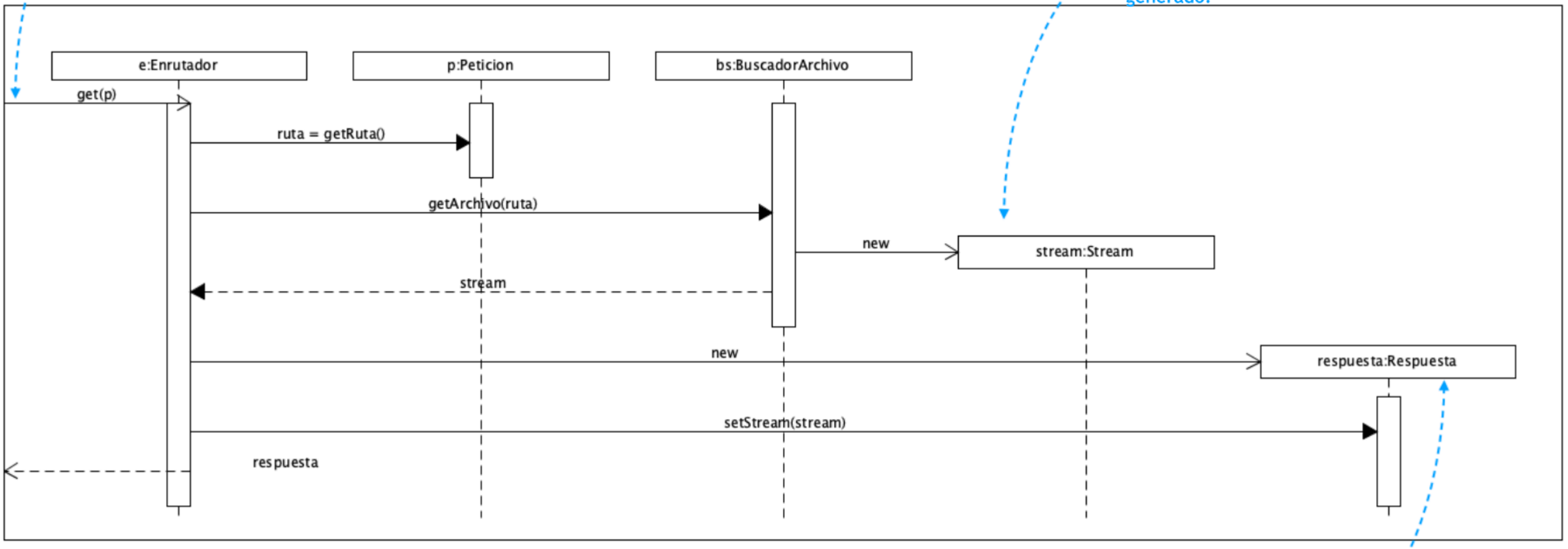
1. Cuando se recibe una petición para acceder a un archivo
2. Cuando se recibe una petición para acceder a una página
3. Cuando una página tiene que llenar una plantilla
4. Cuando una página tiene que consultar información en la base de datos

A continuación veremos cómo los componentes de la solución colaboran en estos 4 escenarios, utilizando diagramas de secuencia. Los intercambios de mensajes entre componentes no son necesariamente llamados a métodos, pero tal vez nos servirán más adelante para definir los métodos de cada clase.

# Colaboraciones: 1. Recibir una petición por un archivo

La solicitud llega inicialmente desde afuera (de la Interfaz HTTP) y llega al enrutador

Una vez encontramos un archivo, lo vamos a manejar como un *stream* de datos. De esa manera no nos importa si es un archivo en el disco o un archivo generado.



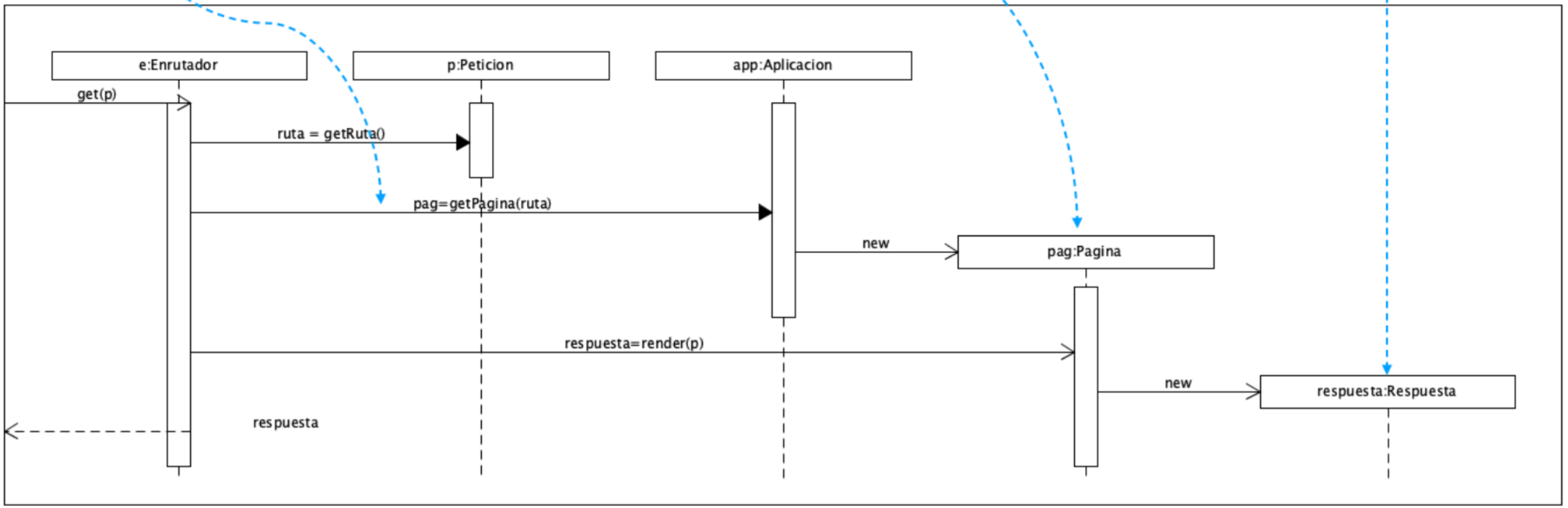

La respuesta es el objeto que encapsula todo lo que hay que enviarle a quien hizo la petición; en particular, la respuesta tiene un *stream* con el archivo resultado.

# Colaboraciones 2: Recibir una petición por una página

El enrutador selecciona la aplicación a donde va la petición

La aplicación selecciona (y construye) la página que debe ser capaz de generar la respuesta.

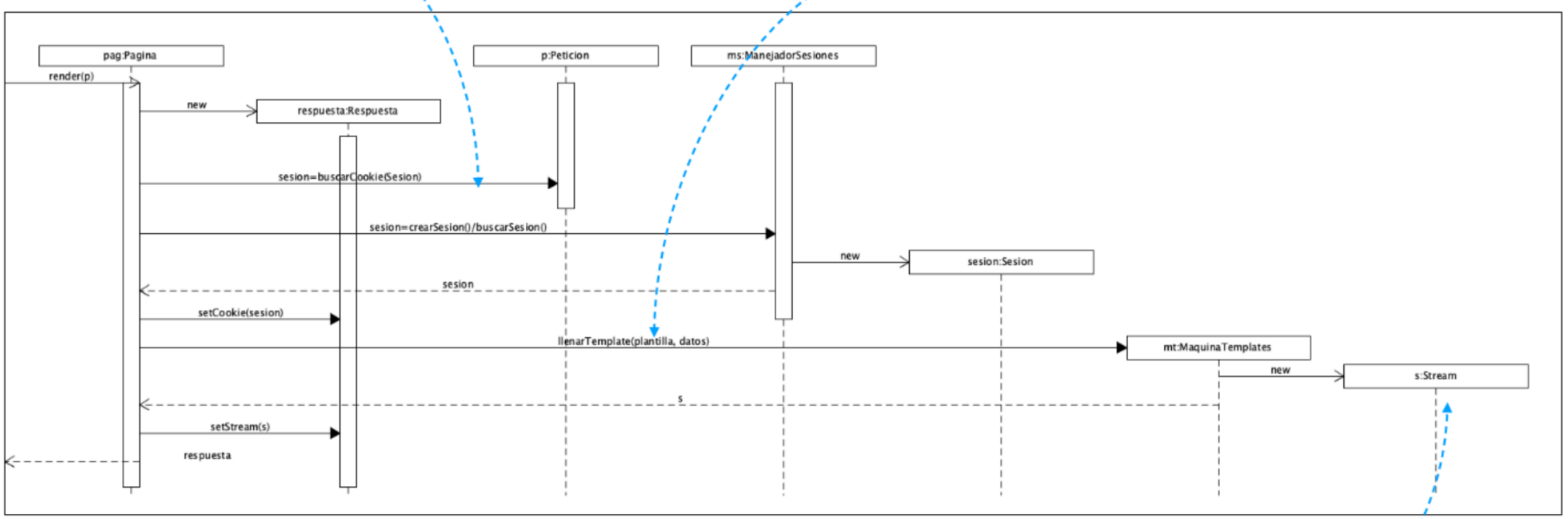
En este caso, es el componente Pagina el encargado de construir la respuesta que se le enviará a quien hizo la petición.



# Colaboraciones 3: Ensamblar una página

La Pagina se asegura de que siempre haya una sesión creada, ayudándose con el ManejadorSesiones

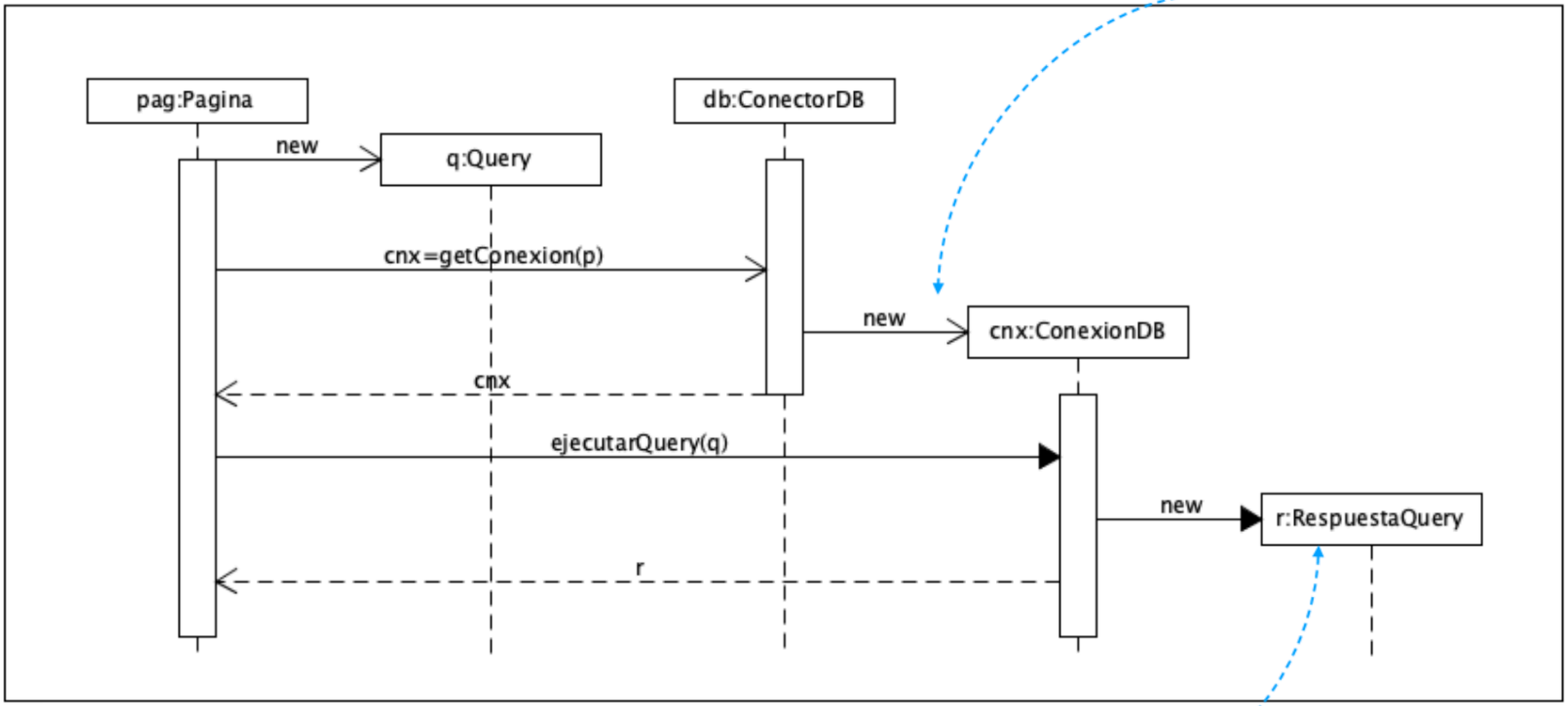
La Pagina se encarga de conseguir los datos que se necesitan para llenar la plantilla y luego le pasa a la MaquinaTemplates la responsabilidad de producir la plantilla llena.




La MaquinaTemplates produce un resultado y lo deja en un stream que luego se dejará dentro de la respuesta. Al ser un stream, no sabemos (y no nos importa) si es un archivo en el disco o es sólo algo en memoria - pero sabemos que tiene la plantilla llena con todo lo que se necesitaba.

# Colaboraciones 4: Ejecutar un *query* sobre la BD

El ConectorDB se encarga de crear las conexiones a la base de datos




La conexión se encarga de ejecutar la consulta y producir un resultado

# Hagamos una validación



¿Existen muchas conexiones desde afuera del vecindario?



¿Hay muchos mensajes de bajo nivel ?



¿ Hay muchas elecciones y caminos alternativos?

# Hagamos una validación



Esto no ocurre en este caso: para cada escenario la cantidad de elementos que participan es relativamente baja.



En este caso la cantidad de mensajes por escenario es relativamente baja: hay intercambio de información de bajo nivel.



En los escenarios explorados no hay muchas alternativas y son todos relativamente lineales.

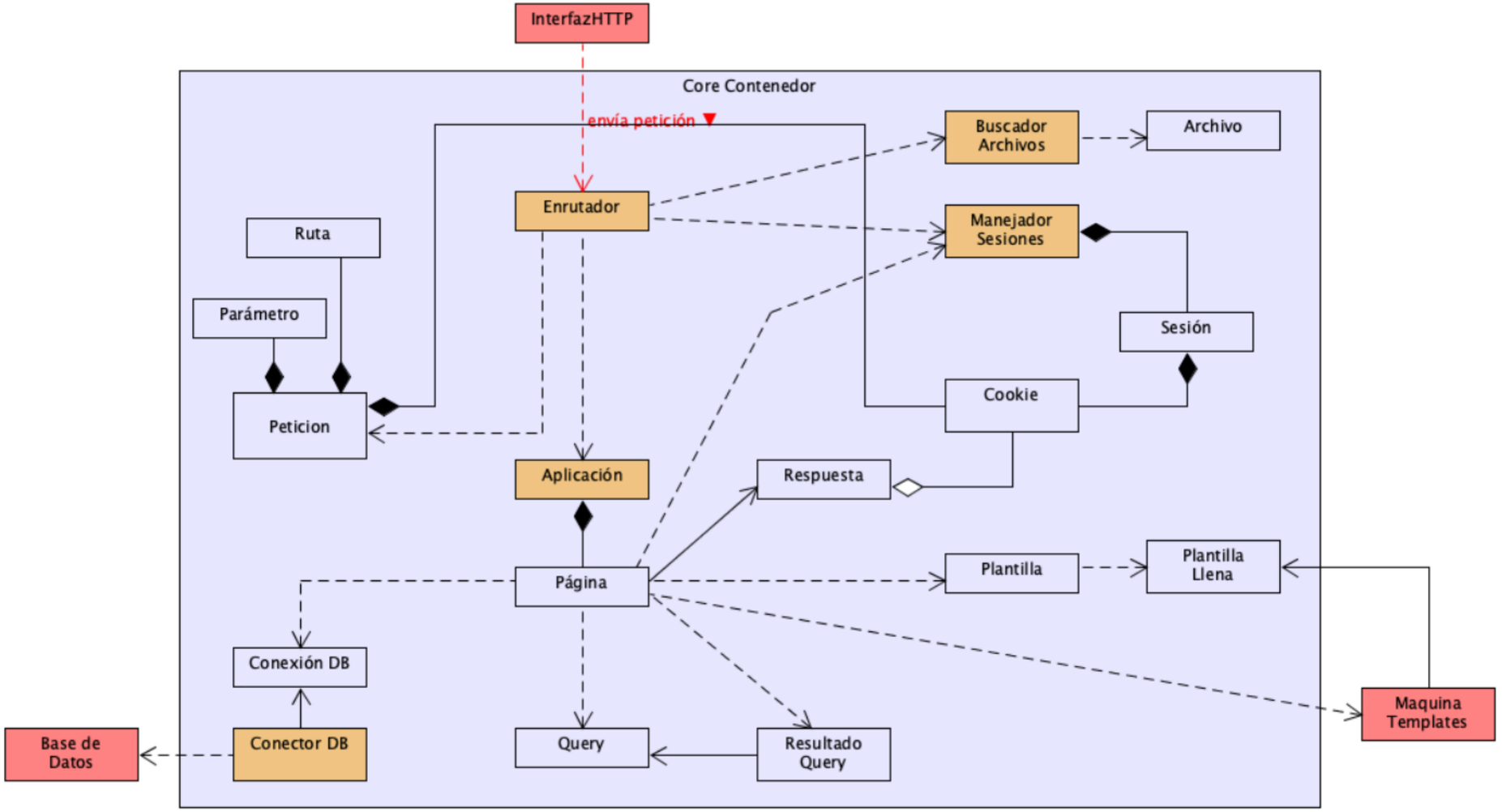


# Finalmente, juntemos todo en la aplicación

# Caso de estudio: actividades

1. Aterrizar cada uno de los componentes candidatos en un elemento implementable o marcarlos para diseñarlos en la siguiente iteración.
    1. Diseñar las clases, con sus atributos y métodos
    2. Diseñar las estructuras de datos para mantener las relaciones entre los objetos
    3. Especificar los contratos (nombres de métodos, tipos de parámetros y retornos)

# Componentes candidatos



# Decisiones iniciales - para simplificar

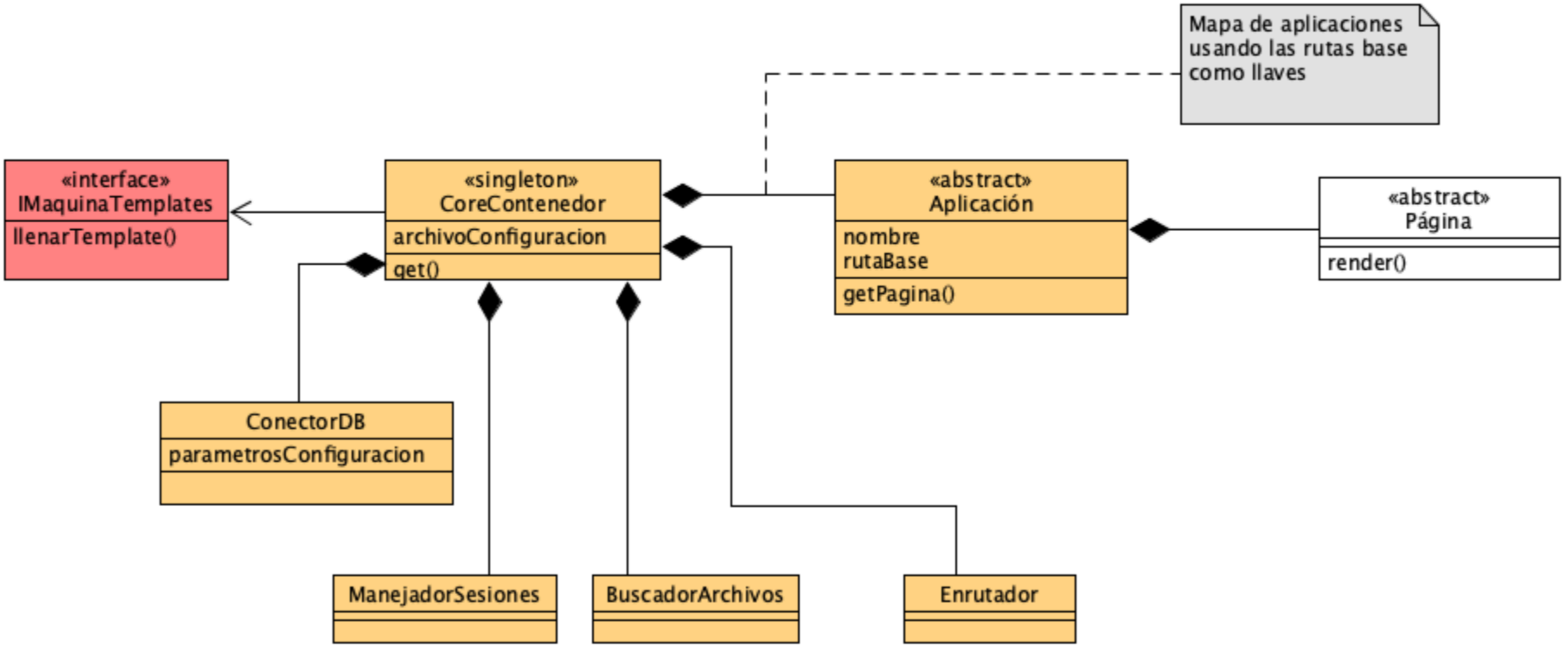
- Una **Plantilla** es un archivo dentro del sistema de archivos, con un formato que se definirá más adelante, cuando se diseñe la Máquina de Templates

- Una **Plantilla Llena** se materializará como un **Stream** (un flujo de datos). De esta manera la Máquina de Templates tendrá libertad para almacenar los templates llenos en archivos o generarlos en memoria. Los **Stream** serán elementos que ya ofrezca el lenguaje ( `java.io.InputStream` )

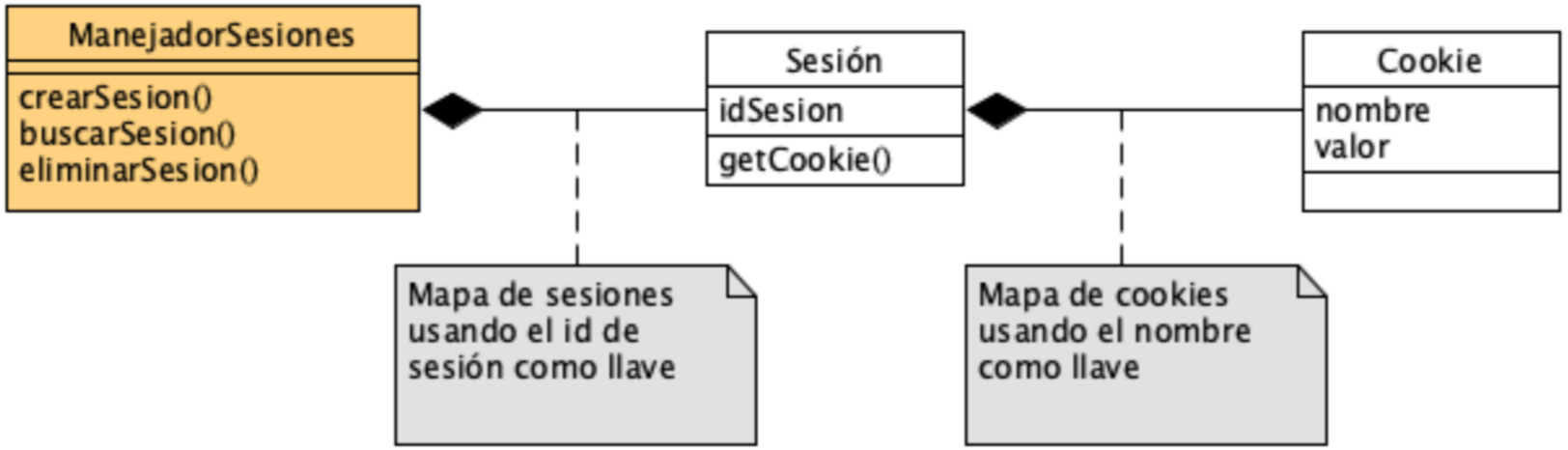
- Los **Archivos** se materializarán también como **Stream**.

- Los elementos de tipo Resultado Query se basarán en lo que ofrezca **`jdbc`** para no repetir cosas que ya son parte de la librería estándar.

# Clases en el Contenedor

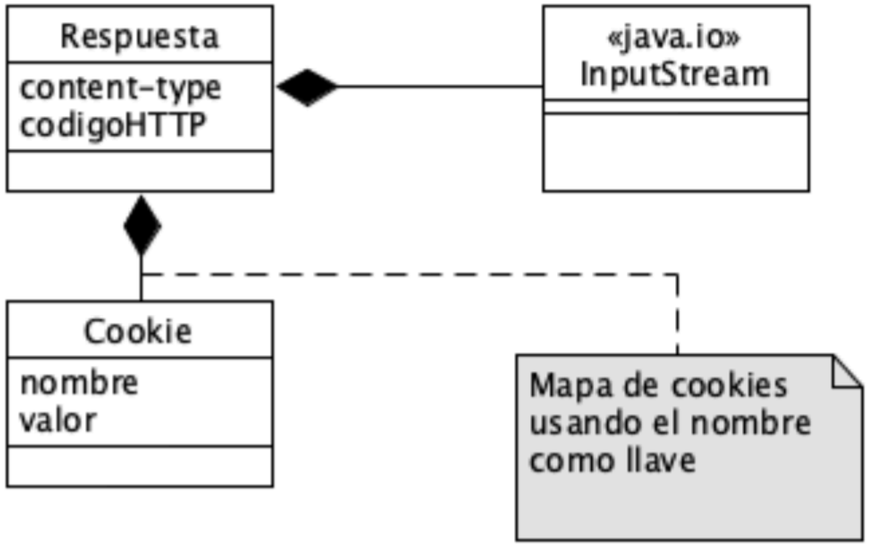


# Manejador de Sesiones



# Peticiones y respuestas



# Componentes pendientes

En este punto ya tenemos bastante claridad sobre la estructura del Core del contenedor de aplicaciones, pero falta diseñar el detalle de los siguientes elementos:

- **Enrutador**, especialmente su estructura interna y la forma en la que decidirá a quién debe dirigirle una nueva solicitud (al Buscador de Archivos o a una Aplicación)
- **Aplicación**, y en particular la forma en la que seleccionará una Página con base en la ruta de la solicitud.
- **ConectorDB** y **ConexionDB**, teniendo en cuenta que deberían reutilizar todos los elementos que fueran posibles de JDBC.
- **BuscadorArchivos**, especialmente en la forma en que buscará los archivos dentro del sistema local a partir de la ruta que se reciba en la petición
- Interfaz de la **MáquinaTemplates**, definiendo en detalle los métodos que ofrecerá para que utilice el core.